

Laporan Final Project Scale Invariant Feature Transform (SIFT)

Muhammad Mishbahul Muflihini
Teknik Informatika
Universitas Darussalam Gontor
Ponorogo, Indonesia
muhammadmishbahulmuflihini10@stud
ent.cs.unida.gontor.ac.id

Abstract— Scale-Invariant Feature Transform (SIFT) merupakan salah satu algoritma ekstraksi fitur klasik yang banyak digunakan dalam bidang visi komputer karena kemampuannya dalam mendeteksi dan mendeskripsikan fitur yang stabil terhadap perubahan skala, rotasi, dan pencahayaan. Pada tugas ini, dilakukan implementasi algoritma SIFT menggunakan bahasa pemrograman Python dan pustaka OpenCV untuk melakukan ekstraksi fitur pada objek bangunan. Hasil eksperimen menunjukkan bahwa penggunaan Difference of Gaussian (DoG) sebagai band-pass filter efektif dalam menonjolkan fitur penting serta membuang noise frekuensi tinggi. Melalui pemanfaatan matriks homografi dan algoritma RANSAC, implementasi ini berhasil menghasilkan panorama yang mulus dengan mengatasi kendala warping citra melalui perluasan kanvas.

Keywords—Keypoint Detection, Difference of Gaussian, Feature Matching, Image Stitching, Panorama.

I. PENDAHULUAN

Visi komputer memerlukan metode ekstraksi fitur yang tangguh untuk mengenali objek dalam berbagai kondisi, seperti perubahan skala dan rotasi. Algoritma Scale-Invariant Feature Transform (SIFT) merupakan solusi klasik yang mengekstraksi fitur lokal distingtif. Tugas ini bertujuan untuk mengimplementasikan alur SIFT secara lengkap, mulai dari pembentukan ruang skala hingga aplikasi praktis dalam pembuatan panorama.

II. DASAR TEORI

SIFT mencapai **invariansi skala** dengan menggunakan piramida citra yang dikaburkan melalui filter Gaussian pada berbagai nilai sigma.

- Keypoint Detection: Menggunakan *Difference of Gaussian* (DoG) sebagai aproksimasi *Laplacian of Gaussian* (LoG) yang lebih efisien untuk menemukan titik-titik ekstrem (blob).
- Orientation Assignment: Setiap *keypoint* diberikan orientasi berdasarkan gradien lokal untuk mencapai invariansi rotasi.
- **Descriptor**: Representasi fitur dalam bentuk histogram orientasi 4 x 4 blok untuk memastikan deskriptor unik dan stabil.

III. METODOLOGI

Sistem diimplementasikan menggunakan bahasa pemrograman Python 3 dengan pustaka utama OpenCV (cv2)

untuk pemrosesan citra, NumPy untuk operasi matriks, dan Matplotlib untuk visualisasi data. Seluruh kode program, dataset, dan dokumentasi cara penggunaan sistem ini telah diunggah dan dapat diakses secara terbuka melalui repositori GitHub berikut:

Tautan Repositori: <https://github.com/Mystery-World3/SIFT-Panorama-Stitching>

Alur kerja sistem dibagi menjadi tahapan berikut:

1. Pra-pemrosesan Citra:

Citra masukan dimuat menggunakan `cv2.imread` dalam format BGR. Untuk keperluan visualisasi dengan Matplotlib, citra dikonversi ke RGB. Sedangkan untuk proses komputasi SIFT, citra dikonversi ke Grayscale karena algoritma bekerja berdasarkan intensitas piksel.

2. Simulasi Scale-Space dan DoG

Untuk memahami konsep ruang skala, dilakukan simulasi manual:

- Gaussian Blur: Citra dikaburkan dengan variasi kernel Gaussian untuk mensimulasikan pandangan pada jarak yang berbeda.
- Difference of Gaussian (DoG): Diimplementasikan dengan mengurangi dua citra yang telah dikaburkan menggunakan parameter.
- Deteksi Blob Manual: Hasil DoG dinormalisasi ke rentang 0-255 dan dilakukan *thresholding* (ambang batas 200) untuk memvisualisasikan kandidat *blob* atau fitur potensial.

3. Ekstraksi Fitur SIFT

Implementasi utama menggunakan fungsi bawaan `cv2.SIFT_create()`. Fungsi `detectAndCompute` digunakan untuk mendapatkan:

- Keypoints: Koordinat (x, y), ukuran (skala), dan sudut (orientasi) dari fitur.
- Descriptors: Vektor fitur 128-dimensi yang merepresentasikan histogram orientasi gradien di sekitar *keypoint*.

4. Pencocokan Fitur (Feature Matching)

Pencocokan dilakukan antara dua citra menggunakan Brute-Force Matcher (cv2.BFMatcher) dengan norma L2. Metode K-Nearest Neighbors (KNN) dengan $k=2$ digunakan untuk mendapatkan dua kecocokan terbaik untuk setiap fitur. Validasi dilakukan menggunakan Lowe's Ratio Test dengan rasio ambang batas 0.75 untuk mengeliminasi *false positives*.

5. Image Stitching dan Perluasan Kanvas

Tahapan pembentukan panorama meliputi:

- Estimasi matriks Homografi (H) menggunakan RANSAC dengan toleransi reprojection error sebesar 5.0.
- Canvas Expansion: Menghitung sudut-sudut citra setelah transformasi untuk menentukan dimensi kanvas baru (lebar dan tinggi panorama total).
- Translasi: Membuat matriks translasi untuk menggeser citra agar tidak ada bagian yang terpotong (koordinat negatif) saat proses *warping*.
- Penggabungan citra menggunakan cv2.warpPerspective dengan matriks gabungan (Translasi x Homografi).

IV. HASIL DAN ANALISIS

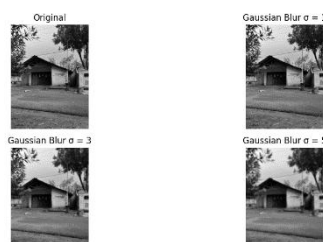
A. Deskripsi Dataset



Gambar 1 & 2 Dataset

Eksperimen ini menggunakan data primer berupa dua buah citra digital yang diambil menggunakan kamera *smartphone*. Objek yang diamati adalah struktur bangunan dengan arsitektur atap limasan dan pagar tembok bermotif.

B. Konstruksi Scale-Space



Gambar 3 Visualisasi Gaussian Blur pada berbagai nilai sigma.

Langkah pertama algoritma SIFT adalah membangun *Scale-Space*. Pada **Gambar 3**, citra asli dikaburkan (*blur*) menggunakan filter Gaussian dengan nilai sigma yang meningkat. Semakin besar nilai sigma, citra semakin kabur, menyimulasikan objek yang dilihat dari jarak jauh. Hal ini memastikan bahwa fitur yang diekstraksi nantinya bersifat *scale-invariant* (dapat dikenali baik pada ukuran besar maupun kecil).

C. Difference of Gaussian (DoG) dan Deteksi Blob



Gambar 4 Proses pembentukan citra Difference of Gaussian (DoG).



Gambar 5 Kandidat Blob hasil thresholding dari DoG.

Untuk mendeteksi lokasi fitur secara efisien, digunakan metode Difference of Gaussian (DoG). Seperti terlihat pada Gambar 4, DoG diperoleh dengan mengurangkan dua citra yang telah di-blur dengan sigma berbeda. DoG bertindak sebagai *band-pass filter* yang menonjolkan tepi dan sudut. Hasilnya kemudian di-*threshold* (ambang batas) seperti pada Gambar 5, di mana titik-titik putih (blob) merepresentasikan kandidat lokasi *keypoint* yang potensial.

D. Lokalisasi Keypoint dan Orientasi



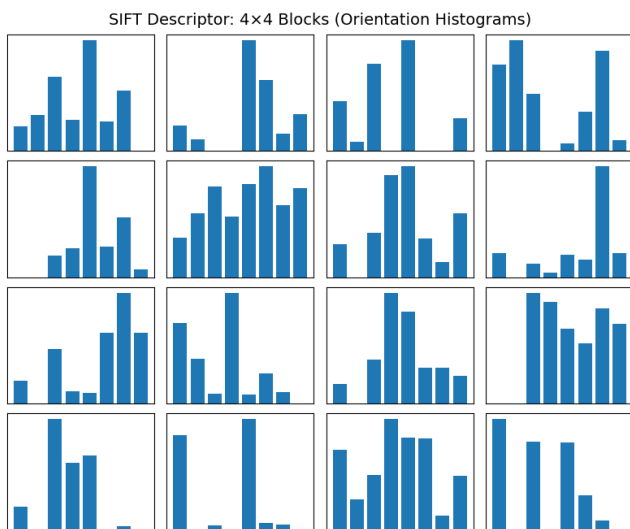
Gambar 6 Visualisasi Keypoint SIFT



Gambar 7 Detail orientasi pada salah satu keypoint.

Setelah kandidat blob disaring (menghilangkan fitur yang berada di tepi/edge atau memiliki kontras rendah), diperoleh himpunan *keypoint* final seperti pada Gambar 6. Lingkaran warna-warni menunjukkan skala fitur tersebut ditemukan. Pada Gambar 7, diperlihatkan atribut orientasi. Panah merah menunjukkan arah gradien dominan di sekitar titik tersebut. Informasi orientasi ini disimpan agar deskriptor nantinya dapat diputar menyesuaikan arah objek, memberikan sifat *rotation-invariance*.

E. Deskriptor Fitur *SIFT*



Gambar 8 Representasi Histogram Orientasi (Descriptor) dari blok 4x4.

Setiap *keypoint* diubah menjadi vektor angka yang disebut *Descriptor*. Gambar 8 memvisualisasikan isi deskriptor tersebut berupa histogram orientasi dari area 16 x 16 piksel (dibagi menjadi 4x4 sub-blok) di sekitar *keypoint*. Pola histogram ini sangat unik untuk setiap bagian gambar (misalnya, pola genteng akan memiliki histogram berbeda dengan pola lantai), memungkinkan pembedaan fitur yang sangat akurat.

F. Pencocokan Fitur (*Feature Matching*)



Gambar 9 Hasil pencocokan fitur menggunakan Ratio Test.

Gambar 9 menampilkan hasil pencocokan antara Citra 1 dan Citra 2 menggunakan metode *Brute-Force Matcher*. Garis warna-warni menghubungkan titik fitur yang identik. Penerapan Lowe's Ratio Test (dengan rasio 0.75) terbukti efektif membuang kecocokan yang salah. Terlihat bahwa fitur-fitur kompleks seperti pola pada pagar tembok (wajik) dan tekstur genteng berhasil dipasangkan dengan tepat meskipun terdapat pergeseran sudut pandang.

G. Hasil Akhir Panorama (*Image Stitching*)



Gambar 10 Proses Stitching: Input, Matches, dan Hasil Akhir Panorama.

Hasil akhir integrasi sistem diperlihatkan pada Gambar 10. Berdasarkan titik-titik *matches* yang valid, dihitung matriks transformasinya (Homografi).

- Analisis Warping: Citra kanan ditransformasikan agar perspektifnya sejajar dengan citra kiri.
- Expanded Canvas: Area hitam di sekeliling panorama menunjukkan hasil teknik *Canvas Expansion*. Tanpa teknik ini, bagian citra yang koordinat transformasinya negatif akan terpotong.
- Kualitas: Penggabungan terlihat mulus pada area sambungan (tengah), membuktikan bahwa fitur SIFT yang diekstraksi memiliki akurasi posisi sub-piksel yang baik.

V. PENJELASAN PRIBADI

A. Pemahaman konsep *SIFT*

Sebelum mengerjakan *Final Project* ini, pemahaman saya mengenai deteksi fitur dalam visi komputer sangat terbatas pada konsep sederhana seperti deteksi tepi (*edge detection*) atau sudut (*corner detection*) menggunakan algoritma Harris. Saya beranggapan bahwa komputer mengenali objek hanya dengan membandingkan nilai piksel secara langsung.

Namun, melalui implementasi SIFT (*Scale-Invariant Feature Transform*), pandangan saya berubah total. Saya kini memahami bahwa sebuah fitur yang baik itu fitur harus tetap dikenali meskipun objek diperbesar, diperkecil, atau diputar. Saya belajar bahwa SIFT mencapai hal ini melalui dua mekanisme utama:

1. Ruang Skala (Scale-Space): SIFT tidak melihat gambar hanya pada satu resolusi. Dengan menggunakan piramida *Difference of Gaussian* (DoG), algoritma ini mencari titik-titik ekstrem yang stabil di berbagai level keburaman (*blur*). Ini menjelaskan mengapa fitur pada rumah dan pagar tembok bermotif dalam final project ini tetap terdeteksi meskipun saya mengambil foto dari jarak yang berbeda.
2. Deskriptor Orientasi: SIFT tidak sekadar menyimpan koordinat (x,y), melainkan menyimpan "sidik jari" berupa histogram gradien di sekitar titik tersebut. Hal inilah yang membuat pencocokan fitur (*feature matching*) menjadi mungkin dilakukan secara akurat, bukan hanya sekadar visual.

B. Kendala dan Solusi

Selama pengerjaan tugas akhir ini, saya tidak mengalami kendala teknis yang signifikan dalam hal penulisan kode atau *error* program. Hal ini dikarenakan seluruh kode sumber (*source code*) utama untuk implementasi SIFT dan penyambungan citra (*image stitching*) telah disediakan oleh dosen pengampu mata kuliah sebagai bahan pembelajaran.

Tantangan utama yang saya hadapi lebih kepada memahami logika dan struktur kode yang diberikan tersebut. Kode yang disediakan menggunakan berbagai fungsi matematika tingkat lanjut dari pustaka OpenCV dan NumPy, seperti penggunaan `cv2.findHomography` dengan metode RANSAC serta transformasi perspektif.

C. Perubahan pemahaman sebelum dan sesudah project

Perubahan signifikan yang saya rasakan adalah pemahaman tentang Kompleksitas Komputasi vs Akurasi. Sebelumnya, saya mengira algoritma visi komputer selalu berjalan instan. Namun, saat menjalankan kode SIFT, saya merasakan adanya *delay* waktu komputasi dibandingkan metode yang lebih sederhana. Ini menyadarkan saya bahwa SIFT adalah algoritma yang "berat" secara komputasi karena melibatkan banyak operasi *floating-point* (konvolusi Gaussian berulang, perhitungan magnitudo gradien, dan orientasi). Hal ini mengajarkan saya bahwa dalam rekayasa

perangkat lunak, selalu ada *trade-off* antara ketangguhan (*robustness*) dan kecepatan (*speed*).

D. Pendapat pribadi

Menurut saya, meskipun saat ini era *Deep Learning* (seperti CNN atau Vision Transformers) mendominasi, SIFT tetap memiliki tempat yang krusial. Kelebihan utamanya adalah tidak membutuhkan data latih (*training data*). Untuk kasus seperti Final Project ini, di mana saya hanya memiliki dua foto Rumah dan pagar tembok bermotif, menggunakan *Deep Learning* akan sangat berlebihan dan tidak efisien. SIFT memberikan solusi matematis yang pasti dan dapat dijelaskan (*explainable*), berbeda dengan *black-box* pada jaringan saraf tiruan. Namun, keterbatasannya terlihat jelas pada gambar dengan tekstur rendah (misalnya dinding polos atau langit biru), di mana SIFT gagal menemukan *keypoint* yang cukup.

VI. KESIMPULAN

Berdasarkan implementasi dan analisis yang telah dilakukan, Saya dapat menarik kesimpulan:

1. Algoritma SIFT terbukti efektif dalam mengekstraksi fitur yang invarian terhadap perubahan skala dan rotasi. Penggunaan *Difference of Gaussian* (DoG) berhasil mendeteksi ratusan *keypoint* stabil pada struktur bangunan.
2. Metode *Feature Matching* menggunakan *Lowe's Ratio Test* (threshold 0.75) sangat krusial untuk menghilangkan kecocokan palsu (*false positives*), memastikan hanya pasangan fitur yang benar-benar unik yang digunakan untuk estimasi geometri.
3. Pembuatan panorama memerlukan penanganan geometri khusus. Estimasi matriks Homografi saja tidak cukup; diperlukan teknik perluasan kanvas dan translasi koordinat untuk mencegah terpotongnya informasi visual saat penggabungan citra.

VII. REFERENCES

- [1] Computer Vision Course by Oddy Virgantara Putra
<https://github.com/virgantara/Computer-VisionCourse/tree/master/courses/week07>

VIII. PERNYATAAN PENGGUNAAN AI

Saya menyatakan bahwa penggunaan *Artificial Intelligence* (AI) dalam membantu penyusunan laporan ini. Seluruh analisis, pemahaman konsep, interpretasi hasil, dan penulisan penjelasan pribadi merupakan hasil pemikiran dan pekerjaan saya sendiri. Saya bertanggung jawab penuh atas isi laporan ini.